

ENTERPRISE ARCHITECTURE BLUEPRINT

Cloud Accounting & Business Management Platform

(Codename "Ledger" - Zoho Books-class product)

VOLUME 5 OF 9

INTEGRATION ARCHITECTURE

Banking - Payments - Tax & E-filing - E-commerce - Public API & Webhooks

Field	Value
Document Status	Draft - v0.1 - Phase 5 of 9
Prepared For	caldim
Prepared By	Claude (Enterprise Architecture Assistant)
Date	13 July 2026
Classification	Internal - Pre-Development Planning
Upstream Dependency	Vol. 1 BC-05, BC-06, BC-11, BC-19; Vol. 2 CTX-05 Banking, CTX-06 Payments, CTX-10 Tax, CTX-16 Integration Gateway; Vol. 3 event bus & anti-corruption layer pattern; Vol. 4 banking/tax schemas
Downstream Volumes	Vol. 6 Security Architecture; Vol. 7 Infrastructure & Deployment; Vol. 8 Testing & QA; Vol. 9 UI/UX & API Design

Table of Contents

1. Introduction & Scope

1.1 Purpose

This volume designs the platform's integrations with the external systems identified in Vol. 1's system context (Section 4 personas, BC-05 Banking, BC-06 Online Payments, BC-11 Tax Compliance, BC-19 Integration & Ecosystem) and the C4 Level 1 diagram (Vol. 2 Section 4): bank feed providers, payment gateways, tax/e-filing services, e-commerce channels, and the platform's own outbound public API and webhook surface for third-party integrators. Every integration in this volume is owned by the Integration Gateway bounded context (CTX-16) or a directly adjacent context (Banking CTX-05, Payments CTX-06, Tax CTX-10) per Vol. 2's context map, and every external call crosses through an anti-corruption layer per Vol. 2 Section 2.2.

1.2 Why Integrations Get Their Own Volume

Third-party financial integrations carry disproportionate operational risk relative to their apparent scope: they are the platform's least controllable dependency (rate limits, breaking API changes, downtime outside the platform's control), the most security-sensitive surface (credentials, tokenized payment data), and the hardest to test end-to-end before production. This volume exists to make every integration's failure mode, retry behavior, and data-ownership boundary explicit before implementation begins, rather than discovering them during an incident.

2. Integration Architecture Principles

#	Principle	Implication
I1	Every external integration sits behind an anti-corruption layer (ACL)	No third-party API shape (field names, enums, pagination style) ever leaks past the Integration Gateway module into core domain modules (Vol. 2 Section 3.3); the ACL translates to the platform's own domain events/DTOs.
I2	Prefer webhooks over polling wherever the provider supports it	Reduces latency and API-quota consumption; polling is the documented fallback, not the default (Section 3, 6).
I3	All inbound webhooks are verified before processing	Signature verification (HMAC or provider-specific scheme) is mandatory before a webhook payload touches any domain logic - unverified webhooks are rejected with 401, not silently ignored, so failures are visible in monitoring.
I4	All integrations are idempotent on both inbound and outbound sides	Inbound: webhook processing dedupes on provider event ID. Outbound: every mutating call to a third party carries an idempotency key (Vol. 3 Section 5) so retries after a timeout cannot double-charge or double-submit.
I5	Integration failures degrade gracefully, they do not corrupt financial state	A failed tax-calculation call blocks invoice send with a clear error rather than posting an invoice with an unknown tax amount; a failed bank-feed sync leaves reconciliation state unchanged rather than partially applying transactions.
I6	Outbound event delivery uses the transactional outbox pattern	A domain event and the database write that caused it commit in the same transaction; a separate relay process publishes to Kafka, guaranteeing no event is lost even if Kafka is briefly unavailable (Section 8).

3. Banking Integration

3.1 Provider Strategy

Direct bank-by-bank integration is not viable at MVP scale (thousands of individual bank connections). The platform integrates with a bank-data aggregator (Plaid/Yodlee/TrueLayer-class, region-dependent - see Section 11 vendor evaluation) that itself maintains the individual bank connections, exposing a single normalized API and webhook stream to the platform.

3.2 Account Linking Flow

Bank account linking (OAuth-style aggregator flow)

```
sequenceDiagram
    participant User as Business Owner/Bookkeeper
    participant Web as Web App
    participant GW as Integration Gateway (CTX-16)
    participant Agg as Bank Aggregator (Plaid-class)
    participant Bank as User's Bank

    User->>Web: Click "Connect Bank Account"
    Web->>GW: request link token
    GW->>Agg: create link token (tenant-scoped)
    Agg-->>GW: link token
    GW-->>Web: link token
    Web->>Agg: open hosted link UI (redirect/iframe)
    User->>Bank: authenticate directly with bank (credentials never touch platform)
    Bank-->>Agg: consent granted
    Agg-->>Web: public token
    Web->>GW: exchange public token
    GW->>Agg: exchange for access token
    Agg-->>GW: access token (stored encrypted, Vol.6)
    GW->>GW: create banking.bank_accounts record via Banking module API
```

User bank credentials never pass through the platform's own servers - this is a deliberate security and compliance boundary (formalized in Vol. 6) that also reduces the platform's liability surface.

3.3 Transaction Sync

Mechanism	When Used	Detail
Webhook (preferred)	Provider supports push notifications for new transactions	Aggregator calls the Integration Gateway's webhook endpoint on new transaction availability; Gateway then pulls the transaction batch via API and publishes BankTransactionImported (Vol.2 Section 10)
Scheduled poll (fallback)	Provider/bank connection does not support webhooks, or webhook delivery has failed repeatedly	Poll every 4 hours per Vol.3 Section 9 background job design; poll frequency is configurable per tenant edition (Enterprise tenants may get more frequent polling)

3.4 Anti-Corruption Layer Responsibilities

- Normalize provider-specific transaction categories into the platform's own category taxonomy before they reach the Banking module.
- Normalize currency/amount representation (some providers return minor units, others decimal) into the platform's NUMERIC(19,4) convention (Vol. 4 Section 8).
- Translate provider-specific error codes into the platform's namespaced error codes (Vol. 3 Section 4.3).
- Shield the rest of the platform from provider API version changes - only the ACL adapter needs to change when the provider ships a breaking API update.

4. Payment Gateway Integration

4.1 Provider Strategy

The platform integrates with a small set of tokenizing payment gateways (Stripe-class for card/ACH in US/EU markets, Razorpay-class for India, expandable per Section 11) via the Payments bounded context (CTX-06), which Vol. 2 Section 2.1 already scoped as an independently-deployable service from Release 1 given its inherent dependency on volatile third-party APIs.

4.2 PCI Scope Reduction

The platform never stores raw card numbers. All card capture happens via the gateway's hosted fields or client-side tokenization SDK, so card data goes directly from the customer's browser to the gateway, and the platform only ever handles an opaque payment token/method ID. This keeps the platform at PCI SAQ-A (Vol. 1 Section 10), the lowest self-assessment tier, rather than SAQ-D.

4.3 Payment Link & Auto-Charge Flow

Payment link generation and webhook confirmation

```
sequenceDiagram
    participant AR as Receivables Module
    participant Pay as Payments Service (CTX-06)
    participant GW as Payment Gateway (Stripe-class)
    participant Cust as Customer

    AR->>Pay: request payment link for Invoice INV-0042
    Pay->>GW: create payment intent (idempotency key = invoiceId)
    GW-->>Pay: payment intent + hosted checkout URL
    Pay-->>AR: payment link
    AR->>Cust: send invoice with payment link (Vol.1 VS-01)
    Cust->>GW: completes payment on hosted checkout
    GW-->>Pay: webhook: payment_intent.succeeded (signed)
    Pay->>Pay: verify webhook signature (Principle I3)
    Pay->>Pay: dedupe on gateway event ID (Principle I4)
    Pay->>AR: publish PaymentRecorded (via event bus, Vol.2 Section 10)
    AR->>AR: apply payment to invoice (Vol.2 Section 9.1 flow continues)
```

4.4 Recurring / Auto-Charge

For recurring invoices (Vol. 1 BC-02, Vol. 3 Section 9 scheduled job), the Payments service stores a reusable payment-method token (not the platform's own credential) and initiates a server-side charge on the recurrence date, with the same idempotency-key and webhook-confirmation pattern as Section 4.3. A failed auto-charge triggers a dunning workflow (Automation module, CTX-14) rather than silent failure.

5. Tax & E-Filing Integration

5.1 Build vs. Buy Decision

Field	Detail
Options Considered	(a) Build an in-house tax rate/rules engine, (b) Integrate a specialized tax-calculation vendor (Avalara/TaxJar-class), (c) Hybrid: simple flat-rate calculation built in-house for MVP, vendor integration for multi-jurisdiction complexity in R2/R3
Decision	Hybrid: in-house single-jurisdiction flat/tiered rate calculation for MVP (Vol.1 Section 6 BC-11 MVP scope), with a pluggable tax-provider interface designed from day one so a specialized vendor can be integrated in R2/R3 without changing the Receivables/Payables call sites
Rationale	Tax jurisdiction rules (especially US sales tax nexus and multi-state rates) are notoriously complex and change frequently; building this in-house beyond simple cases is a poor use of engineering effort relative to buying it from a vendor that specializes in exactly this. The pluggable interface (Section 5.2) avoids a costly rework when that vendor integration is added.

5.2 Tax Provider Interface (Pluggable)

Tax module - pluggable calculation interface

```
interface TaxCalculationProvider {
  calculateTax(input: {
    tenantId: string;
    lineItems: LineItem[];
    customerAddress: Address;
    sellerAddress: Address;
    transactionDate: Date;
  }): Promise<TaxBreakdown>;
}

// MVP implementation: FlatRateTaxProvider (in-house, single jurisdiction)
// R2/R3 implementation: AvalaraTaxProvider (vendor-integrated, multi-jurisdiction)
// Selected per-tenant via configuration (Vol.3 Section 10 feature flag pattern)
```

5.3 E-Filing Integrations

Integration	Scope	Pattern
1099-NEC/MISC e-filing (US)	R3 per Vol.1 Section 6	Batch export to IRS-compatible format (IRIS) or a filing intermediary (Tax1099/Yerli-class); asynchronous job, not real-time API call
Sales tax return filing	R3, jurisdiction-dependent	Where the tax vendor (Section 5.1) offers filing-as-a-service, delegate to it; otherwise generate a filing-ready report for manual submission
GST filing (India)	R3	Integration with GSTN-compatible filing API or intermediary, similar batch-export pattern

6. E-Commerce Channel Integration

6.1 Supported Channels & Sync Direction

Channel	Inventory Sync	Order Import	Pattern
Shopify	Platform -> Channel (stock levels)	Channel -> Platform (new orders)	Webhook for orders; scheduled push for stock levels (batched to respect rate limits)
Amazon	Platform -> Channel	Channel -> Platform	Amazon SP-API; polling required for several resource types per API constraints
Walmart	Platform -> Channel	Channel -> Platform	Includes shipment tracking and payout/settlement reconciliation per Vol.1 research (2026 Walmart integration feature)

6.2 Order Import Flow

E-commerce order import to Sales Order/Invoice

```
sequenceDiagram
    participant Ch as E-commerce Channel (Shopify-class)
    participant GW as Integration Gateway (CTX-16)
    participant INV as Inventory Module
    participant AR as Receivables Module

    Ch->>GW: webhook: order created (signed)
    GW->>GW: verify signature, dedupe on channel order ID
    GW->>INV: check/reserve stock for order line items
    INV-->>GW: stock confirmed
    GW->>AR: create Sales Order (channel = Shopify, external ref = order ID)
    AR->>AR: optionally auto-convert to Invoice per tenant configuration
    GW->>Ch: acknowledge webhook (200 OK)
    Note over GW,Ch: On stock update elsewhere (Vol.2 StockAdjusted event), Gateway pushes updated stock levels back to Channel (rate-limited batch)
```

6.3 Conflict Handling

- If a channel order references a SKU not found in the platform's item master, the order is imported in a Needs Review state rather than silently dropped or auto-created as a new item.
- If stock is insufficient at import time, the order still imports (the sale already happened on the channel) but flags a backorder/oversell warning for the Inventory Manager (PER-06).
- Channel-side price/tax figures are treated as informational; the platform recalculates tax via the Tax module (Section 5) as the authoritative figure for financial posting.

7. Public API & Webhook Platform (Outbound)

This is the platform acting as the upstream integration provider for third-party developers and partner apps - the mirror image of Sections 3-6. It builds directly on Vol. 3 Section 5's API conventions.

7.1 Outbound Webhook Delivery

Aspect	Design
Subscription model	Tenant/developer registers a webhook URL per event type (subset of Vol.2 Section 10's domain event catalog, e.g. invoice.sent, payment.recorded)
Signing	Every outbound webhook payload is signed with an HMAC secret unique per subscription, verifiable by the receiving third party (mirrors Principle I3 for inbound)
Delivery guarantee	At-least-once, with exponential backoff retry (up to 24 hours) and a dead-letter queue for permanently failing endpoints, surfaced in a developer-facing delivery log
Ordering	Best-effort ordering per aggregate (same partitioning as the internal event bus, Vol.3 Section 8.1); receivers must not assume strict global ordering

7.2 Public API Rate Limiting

Rate limits are enforced at the API Gateway (Vol. 2 Section 5) using the Redis-backed counters described in Vol. 3 Section 7.1, tiered by edition (Vol. 1 Section 9): Standard/Professional get a shared default quota, Enterprise tenants get a higher or dedicated quota. Limit responses follow standard 429 + Retry-After conventions.

8. Integration Resilience Patterns

8.1 Transactional Outbox Pattern

Outbox pattern - reliable event publishing

```

flowchart LR
    A[Domain service writes entity change + outbox row in ONE DB transaction] --> B[(Postgres)]
    B --> C[Outbox Relay Process - polls or uses logical replication/CDC]
    C --> D[{Kafka - Vol.3 Section 8.1 topics}]
    C --> E[Mark outbox row published]
  
```

This guarantees that a committed domain event (e.g. InvoiceSent) is never lost even if Kafka is briefly unreachable, and that no event is published for a database write that ultimately rolled back - both failure modes that a naive "publish then write" or "write then publish" approach would allow.

8.2 Circuit Breaker for Outbound Calls

State	Behavior
Closed (normal)	Calls to the external provider proceed normally; failures are counted in a rolling window
Open (tripped)	After a failure-rate threshold is crossed, calls fail fast without hitting the provider, protecting both the platform (no thread/connection exhaustion) and the already-struggling provider from additional load
Half-open (probing)	After a cooldown period, a limited number of calls are allowed through to test recovery before fully closing the circuit again

8.3 Retry Policy Summary

Integration	Retry Strategy	Max Attempts
Outbound webhook delivery (Section 7.1)	Exponential backoff, starting at 30s	Retries for up to 24h, then dead-letter
Bank aggregator API calls (Section 3)	Exponential backoff with jitter	5 attempts, then flagged for manual review
Payment gateway calls (Section 4)	No automatic retry on charge creation (to avoid double-charge risk) - relies on idempotency key + gateway's own idempotent handling instead	1 attempt; failure surfaces to user immediately
Internal Kafka consumer processing (Vol.3 Section 8.2)	Exponential backoff	5 attempts, then dead-letter topic

9. Integration Gateway Service Architecture (CTX-16)

Component detail for the Integration Gateway, extending Vol. 2 Section 6's C4 Level 3 diagrams to this context, which Vol. 2 Section 2.1 already scoped as an independently-deployable service.

C4 Level 3 - Integration Gateway components

```

flowchart TB
    subgraph GW[CTX-16 Integration Gateway Service]
        InWebhook[Inbound Webhook Receiver - signature verification]
        OutWebhook[Outbound Webhook Dispatcher - Section 7.1]
        BankAdapter[Bank Aggregator Adapter - ACL, Section 3]
        EcomAdapter[E-commerce Channel Adapters - ACL, Section 6]
        CircuitBreaker[Circuit Breaker / Retry Layer - Section 8]
        Outbox[Outbox Relay - Section 8.1]
        CredStore[Encrypted Credential Store - tokens/secrets, Vol.6]
    end
    InWebhook --> BankAdapter
    InWebhook --> EcomAdapter
    BankAdapter --> CircuitBreaker
    EcomAdapter --> CircuitBreaker
    CircuitBreaker --> ExtAPIs[External Provider APIs]
    BankAdapter --> CredStore
    EcomAdapter --> CredStore
    Outbox --> OutWebhook
    Outbox --> Bus[Internal Event Bus]
  
```

10. Third-Party Vendor Evaluation Matrix

This is a starting evaluation frame, not a final vendor selection - final selection depends on target-market confirmation (Vol. 1 Section 13.2 open question on countries) and commercial negotiation outside this document's scope.

Category	Candidate Vendors (region-dependent)	Key Evaluation Criteria
Bank data aggregation	Plaid, Yodlee, TrueLayer, Salt Edge	Bank coverage in target markets, webhook support, pricing per connected account, data normalization quality
Payment gateway	Stripe, Razorpay, Adyen, Braintree	PCI SAQ-A compatibility (Section 4.2), supported payment methods per market, payout/settlement reporting quality
Tax calculation	Avalara, TaxJar, Vertex	Jurisdiction coverage, filing-as-a-service availability (Section 5.3), API latency (blocks invoice send, Vol.1 Section 2.4 P95 target)
E-filing (1099/GST)	Tax1099, Yerli, direct IRS IRIS, GSTN-certified providers	Direct government API access vs. intermediary, batch vs. real-time filing

11. Traceability & Next Phase

11.1 Traceability to Volumes 1-4

Vol. 5 Artifact	Traces To
Section 3 Banking Integration	Vol. 1 BC-05; Vol. 2 CTX-05; Vol. 4 banking.bank_transactions schema
Section 4 Payment Gateway Integration	Vol. 1 BC-06; Vol. 2 CTX-06 (scoped as independent service, Section 2.1)
Section 5 Tax & E-filing	Vol. 1 BC-11; Vol. 2 CTX-10
Section 6 E-commerce	Vol. 1 BC-19 (Walmart integration research finding)
Section 7 Public API/Webhooks	Vol. 3 Section 5 API conventions; Vol. 2 Section 10 event catalog (outbound subset)
Section 8 Resilience Patterns	Vol. 3 Section 8.2 idempotent consumers; Vol. 1 Principle P4 (auditability - outbox guarantees no lost financial events)

11.2 Open Questions for Sign-off

1. Confirm target markets (Vol.1 Section 13.2 Q1) to finalize which bank aggregator and tax vendor candidates (Section 10) are actually in-region.
2. Confirm whether a payment gateway aggregator/orchestration layer (to support multiple gateways per region without duplicating integration work) is worth the added complexity at MVP, or whether a single gateway per initial market is acceptable.
3. Confirm risk tolerance for the Section 5.1 hybrid tax approach - is flat-rate MVP tax calculation acceptable to launch with, or does the first target market require multi-jurisdiction accuracy from day one?

11.3 Next Phase

On approval, Phase 6 begins: Security Architecture (Volume 6) - threat model, IAM design, encryption/credential-storage standards (referenced but not detailed in this volume's Section 9 Credential Store), and compliance control mapping (SOC2/GDPR/PCI) covering both the core platform and every integration surface defined here.